25 AUGUST 2026

Coding Café

Dev-Tools I - the terminal

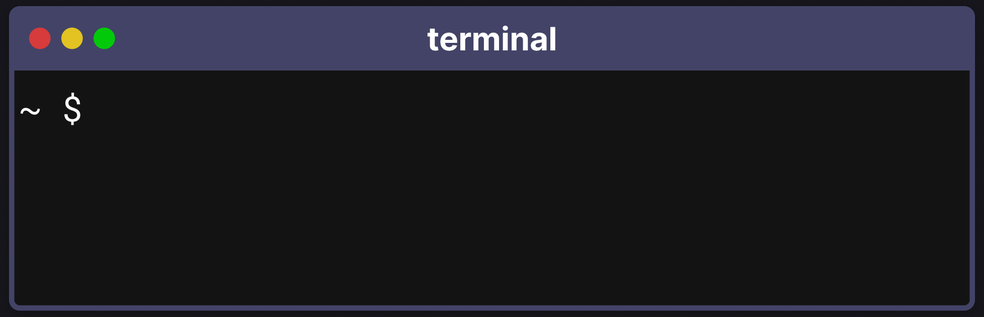
Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

The window your machine actually listens through.

A window shows you what somebody (a software dev) decided to show you.
The shell shows you everything, and lets you say anything back.

Every server you will ever touch has a terminal (command line interface) and no desktop (graphical user interface).
Git, pip and most tools you use this term are terminal-first.

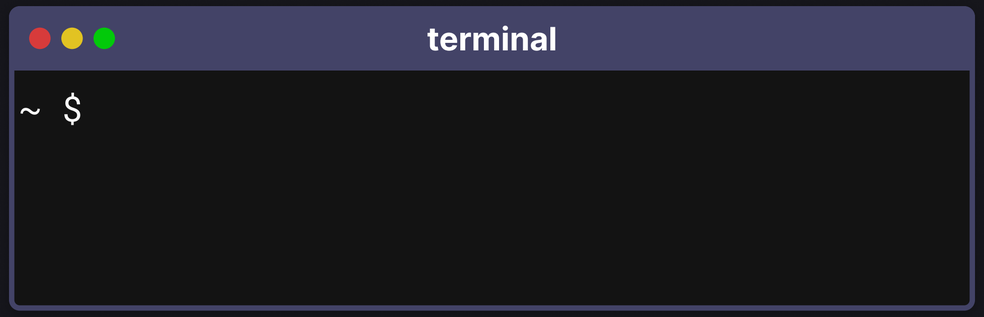


Getting one open.

- macOS / Linux - Terminal app, already installed
- Windows - install WSL from Powershell (refer to email instruction), then open the Ubuntu terminal with `wsl`

WSL (Windows Subsytem for Linux) is a real Linux environment running on Windows without doing a full install.

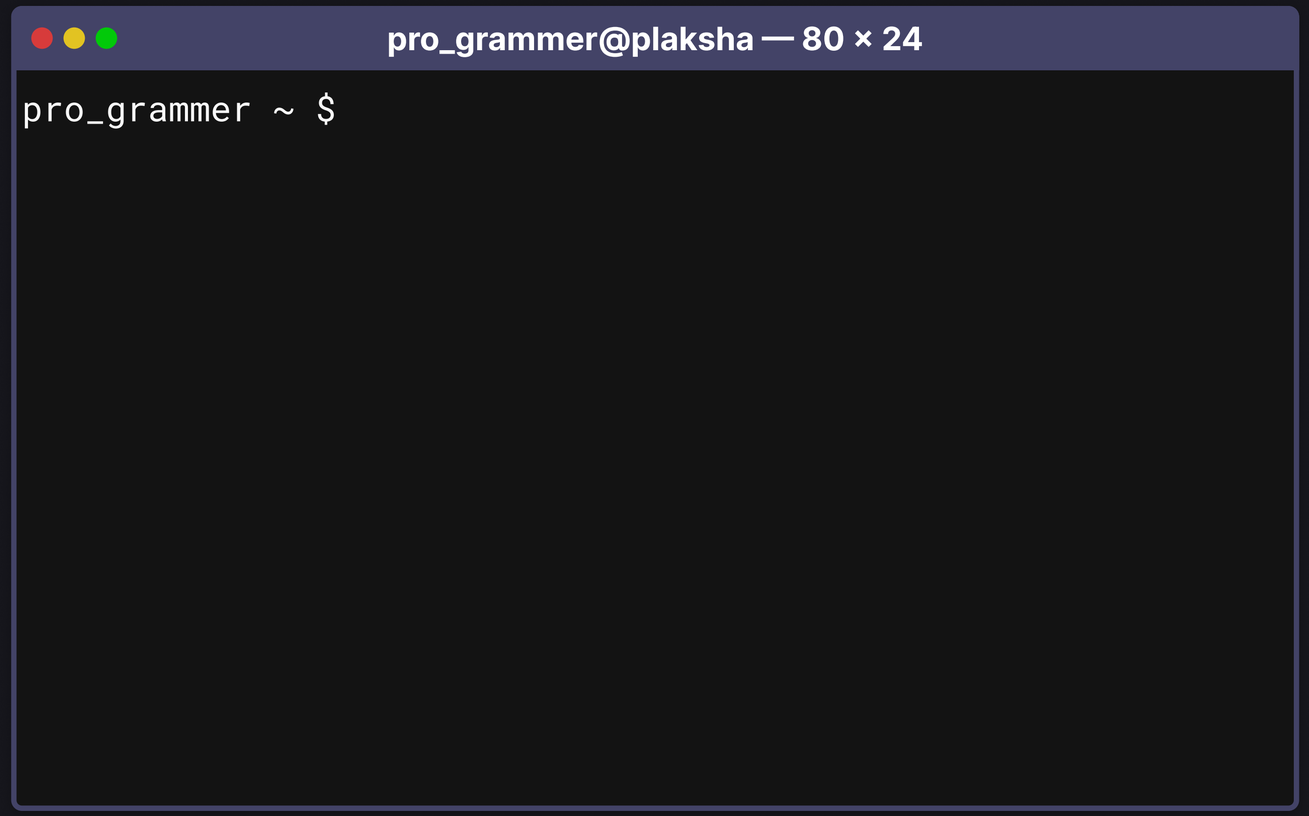
It is why everyone can type the same thing.



Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

Reading the prompt.

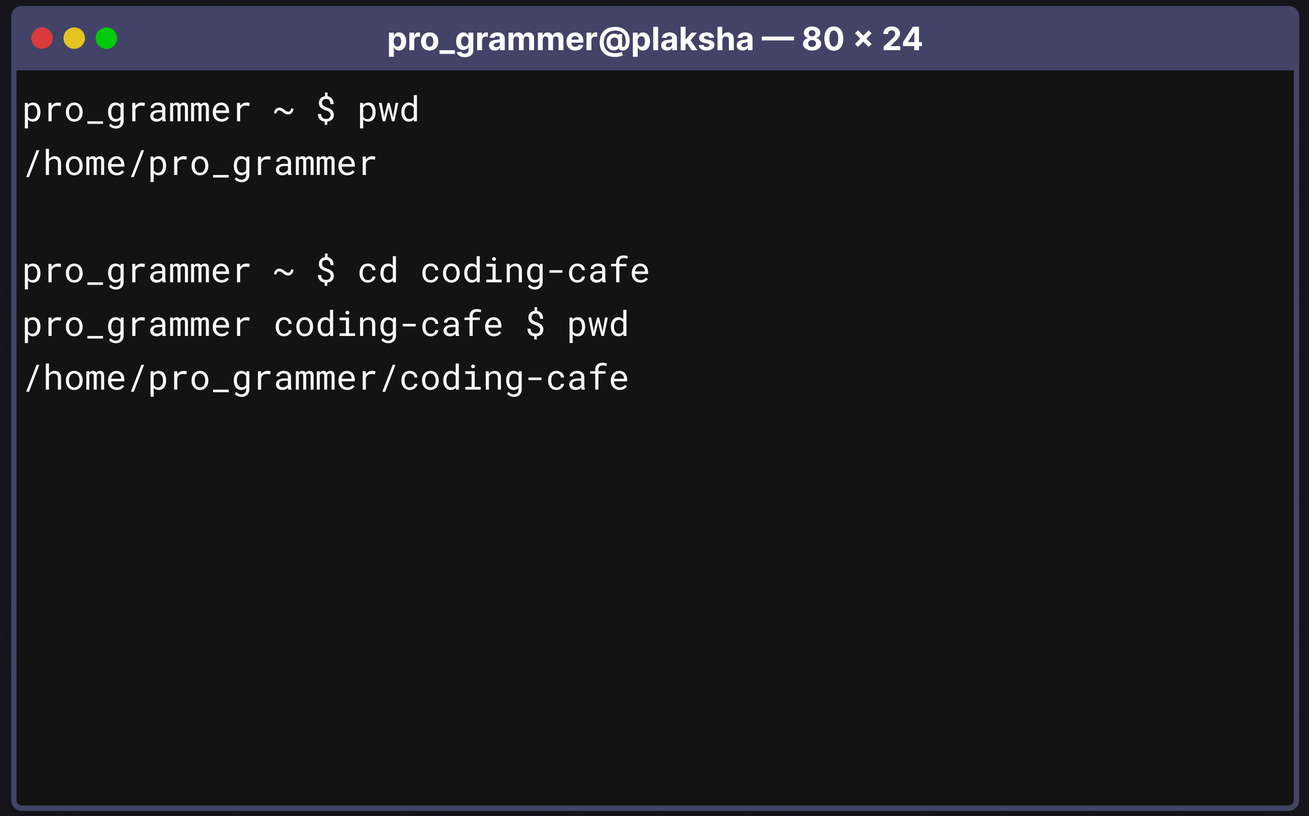


what you see	what it means
pro_grammer	who you are (the user)
~	where you are; the path (~ means home)
$ or %	the shell is ready

You type after the $ or %.

Nothing happens until you press Enter.

Where am I?



pwd - present working directory

This tells you where you are in your machine.
The shell is always standing somewhere.
Every command you type runs from wherever that is.

When something does not work, this is the first thing to check.

What is in here?

pro_grammer@plaksha — 80 × 24

```
pro_grammer coding-cafe $ ls
hello.py   notes.txt   week01

pro_grammer coding-cafe $ ls -a
.   ..   .hidden  hello.py  notes.txt  week01
```

`ls` - list

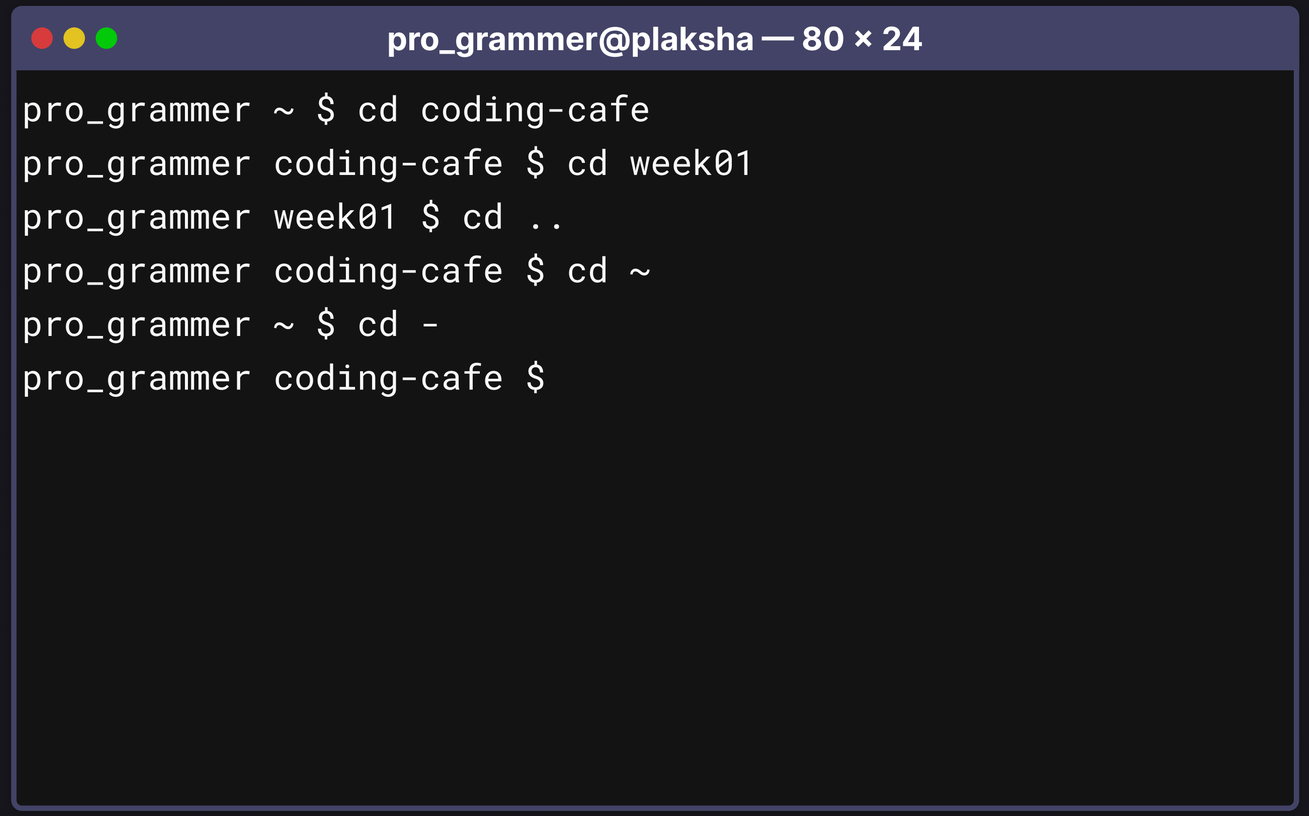
Most programs also have additional flags that can be used to modify the behaviour of the program, like:

- -l the long version: size, and when it last changed
- -a everything, including the dotfiles normally hidden

Flags can be used as short flags (single hyphen and one letter) or long flags (two hyphens and the full word).

Flags stack, you can use multiple together: `ls -la`

Moving around.



`cd` - change directory

`..` up one level (two dots)
`~` your home folder (tilde)
`-` back where you just came from (hyphen)

cd on its own also takes you home.

Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

Two ways to say where something is. The Path.

Absolute path - from the very top. Always means the same

`/home/pro_grammer/coding-cafe/hello.py`

Relative path - from wherever you happen to be standing.

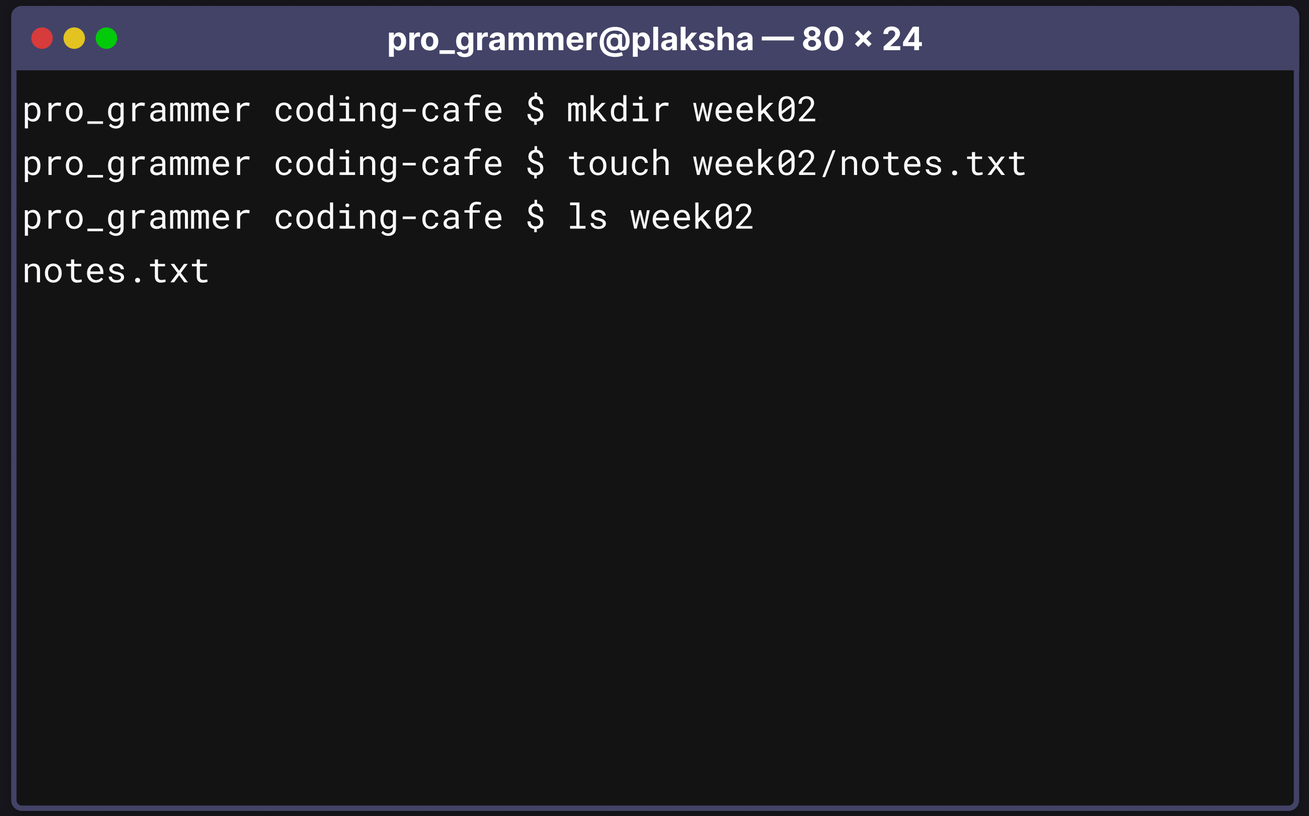
`hello.py` (the same level where you are)

`week01/hello.py` (one level down inside week01 - child level)

`../notes.txt` (one level up - parent level)

symbols	level
.	the current level; here
..	one level up
~	your home folder (assigned by os)
-	the previous location

Making things.



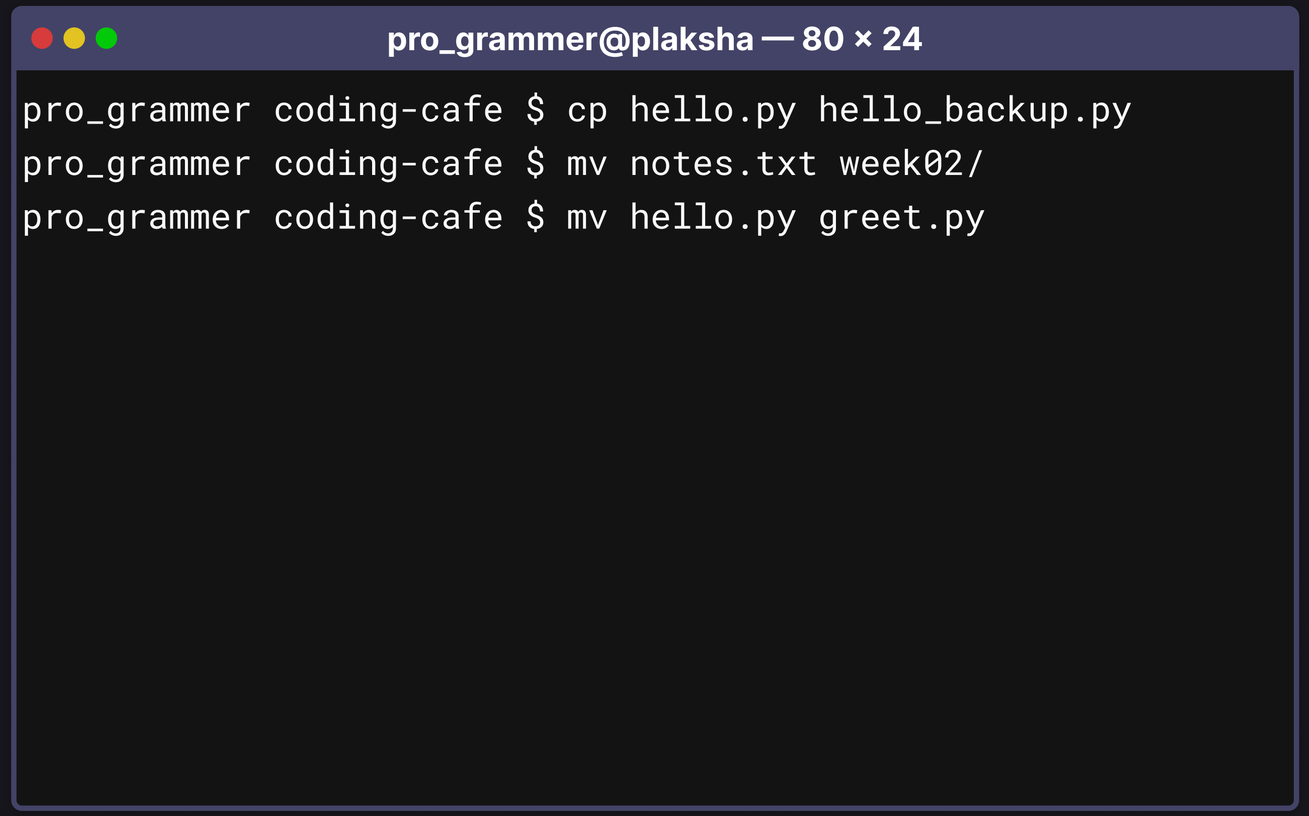
`mkdir` - make a directory
`touch` - make an empty file

`mkdir -p a/b/c` makes the whole chain in one go.
c inside b, inside a

From the program manual:

```
-p, --parents
        make parent directories as needed
```

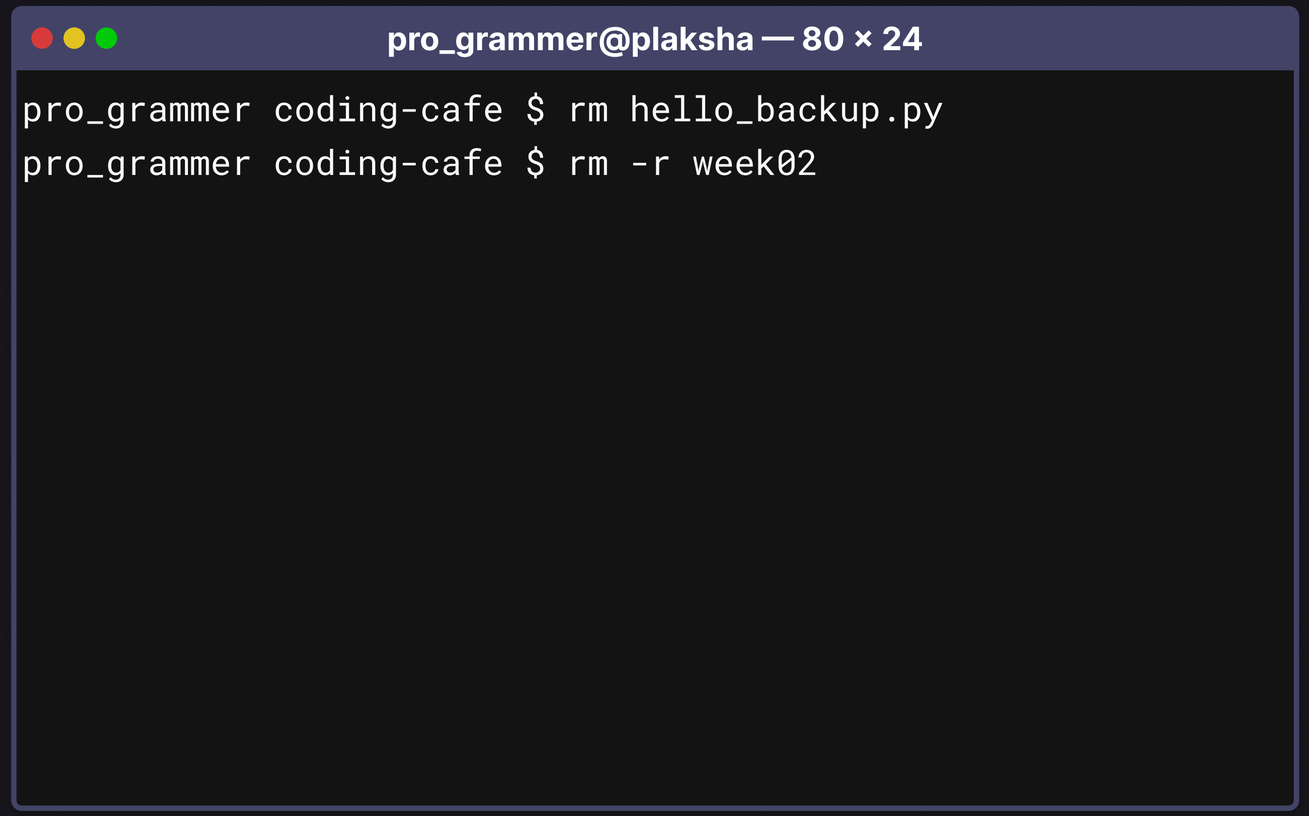
Copying and moving.



`cp` - copy
`mv` - move

mv is also how you rename.
Moving a file to a new name in the same folder is a rename.

Deleting. Carefully.



`rm` - remove
`-r` - and everything inside it

There is no trash can here. Deleted is deleted.

Read the line twice before you press Enter.

Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

How did it know where to look?



Somewhere on this machine there is a file called python3.

You typed six characters. The shell found the exact location in milliseconds.

Before I tell you how, if you were building the shell, how would you solve this?

Three answers that do not work.

1. Keep a variable for every command
   - Then every command needs a $ in front of it, and the shell holds one variable per program on your machine.

2. Search the whole machine, every time
   - Punishingly slow. And if two files share a name, which one wins?

3. Demand that every program lives in one folder
   - Too rigid. An operating system needs more than one place to put things.

A short list, in order.

pro_grammer@plaksha — 80 × 24

```
pro_grammer coding-cafe $ echo $PATH
/usr/local/bin:/usr/bin:/bin

pro_grammer coding-cafe $ which python3
/usr/bin/python3

pro_grammer coding-cafe $ which pip
/usr/bin/pip
```

`PATH` is a list of folders, separated by colons - because every environment variable is only ever text.

The shell walks it left to right and stops at the first match.

This is what “Add python.exe to PATH” meant in Week 1 during installation.

And it is what “command not found” means: the program’s name was not in any of these folders.

`which` tells you which one won.

Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

Running your script.

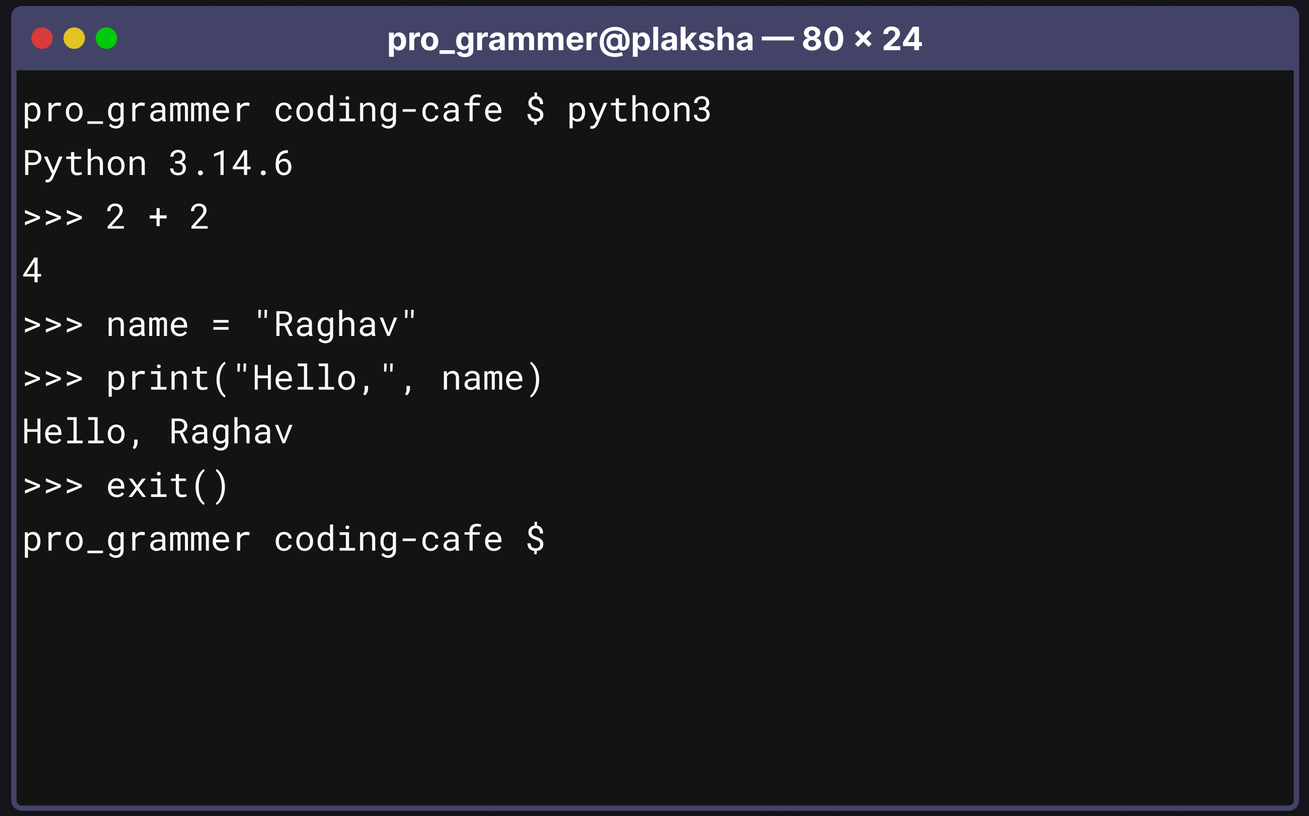


`python3`, then the file.

The path is relative to where you are standing.

If you get “No such file or directory”, run `pwd` and `ls` first - you are probably somewhere else.

Python without a file.



`python3` with no file gives you the REPL.

Read, Evaluate, Print, Loop - the thing from last week.

Good for trying one line. Nothing here is saved.

`exit()` or `Ctrl+D` to leave.

Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

Somebody already wrote it.

pro_grammer@plaksha — 80 × 24

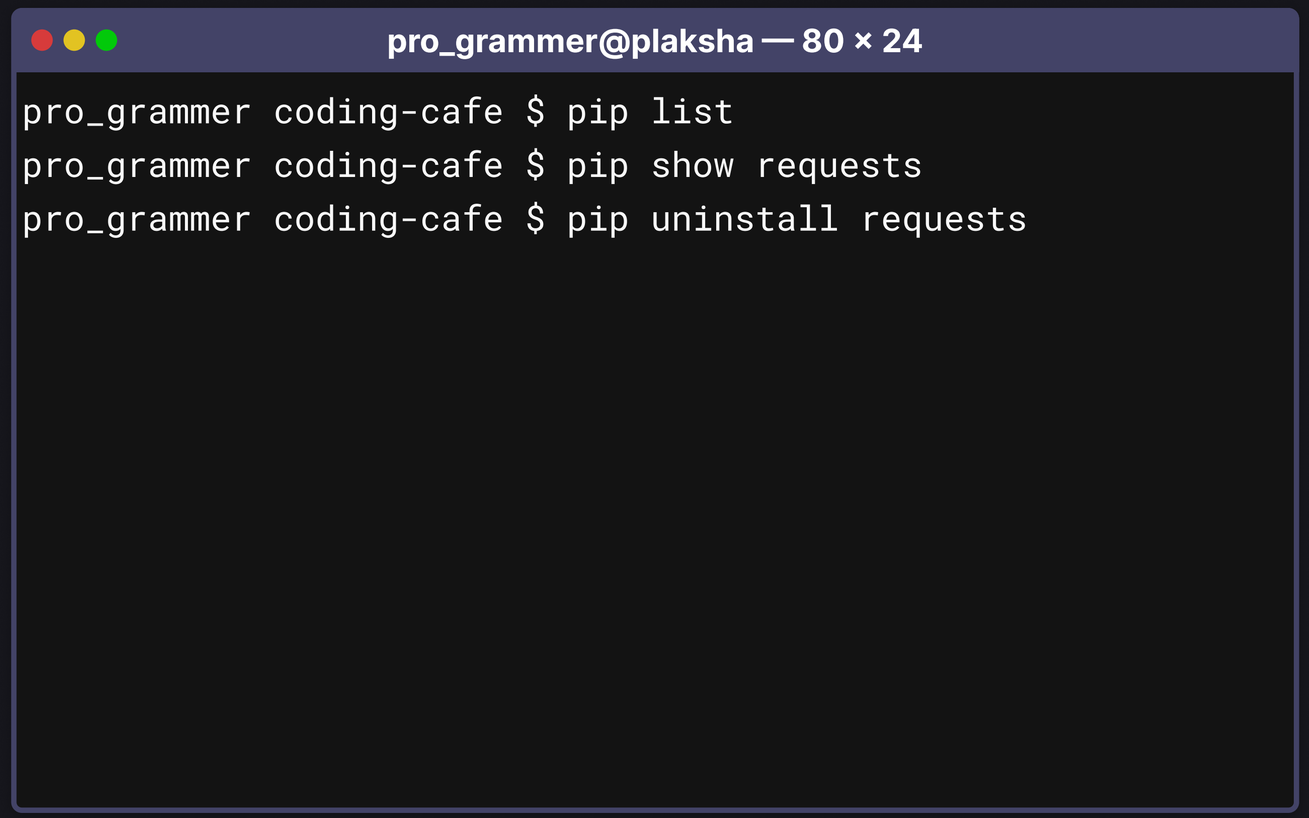
```
pro_grammer coding-cafe $ pip install requests
Collecting requests
  Downloading requests ...
Successfully installed requests
```

`pip` - the package installer for Python

It fetches code other people have written and published, and makes it importable in yours.

We use this properly in later weeks. For now: know it exists, and know this is how you get things.

What do I already have?



`pip list` - everything installed

`pip show <name>` - where it came from, what it needs

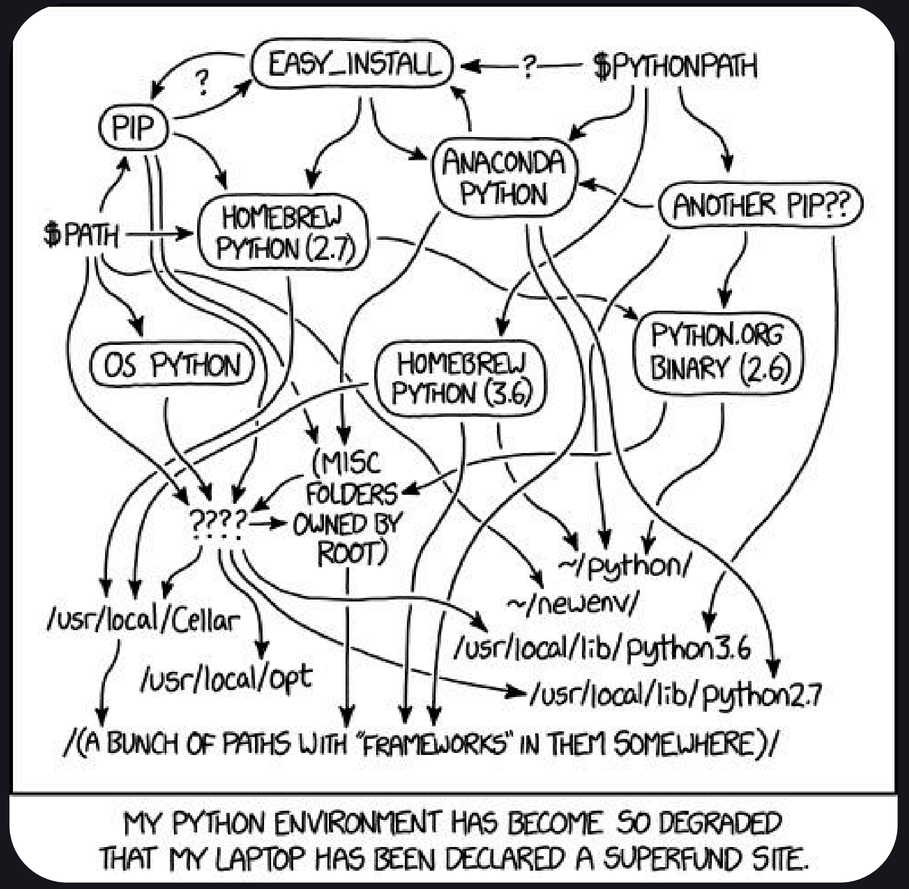
`pip uninstall <name>` - take it back off

Installing everything globally gets messy fast. There is a better way shown on the next slide.

It gets messy.

After enough time of using Python, installing libraries, updating stuff, you will find that your file system can become messy.

This happens when things are piled on top of previous installs, version disparities, and so on.

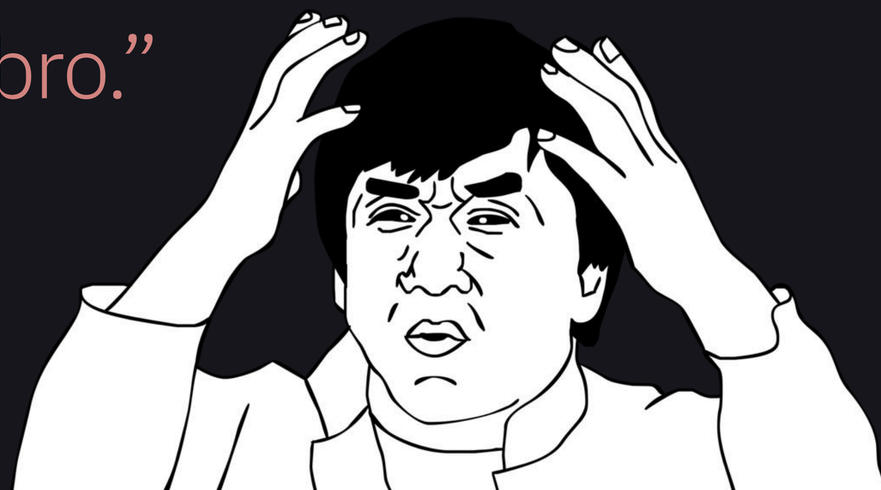


Now imagine sharing it.

Now imagine working with that in any feasible way - finding the right package and the right Python version for a project you are working on with somebody else.

Your machine works. Theirs does not. Nobody knows why.

“It works on my machine bro.”



Isolating each project is a simple way to prevent this.

One box per project.
Its own Python. Its own packages. Its own versions. Only the things that the project needs to run.

Nothing you install for one project can reach into another.

Enter virtual environments!

Keeping projects apart.

pro_grammer@plaksha — 80 × 24

```
pro_grammer coding-cafe $ python3 -m venv .venv
pro_grammer coding-cafe $ source .venv/bin/activate
(.venv) pro_grammer coding-cafe $ pip install requests
Successfully installed requests
(.venv) pro_grammer coding-cafe $ deactivate
pro_grammer coding-cafe $
```

One per project. In Week 6 you will tell Git to ignore it.

A virtual environment is a private box of packages for one project.

build the box

```
python3 -m venv .venv
```

step into it

```
source .venv/bin/activate
```

The (.venv) in your prompt tells you which box you are standing in.

step back out

```
deactivate
```

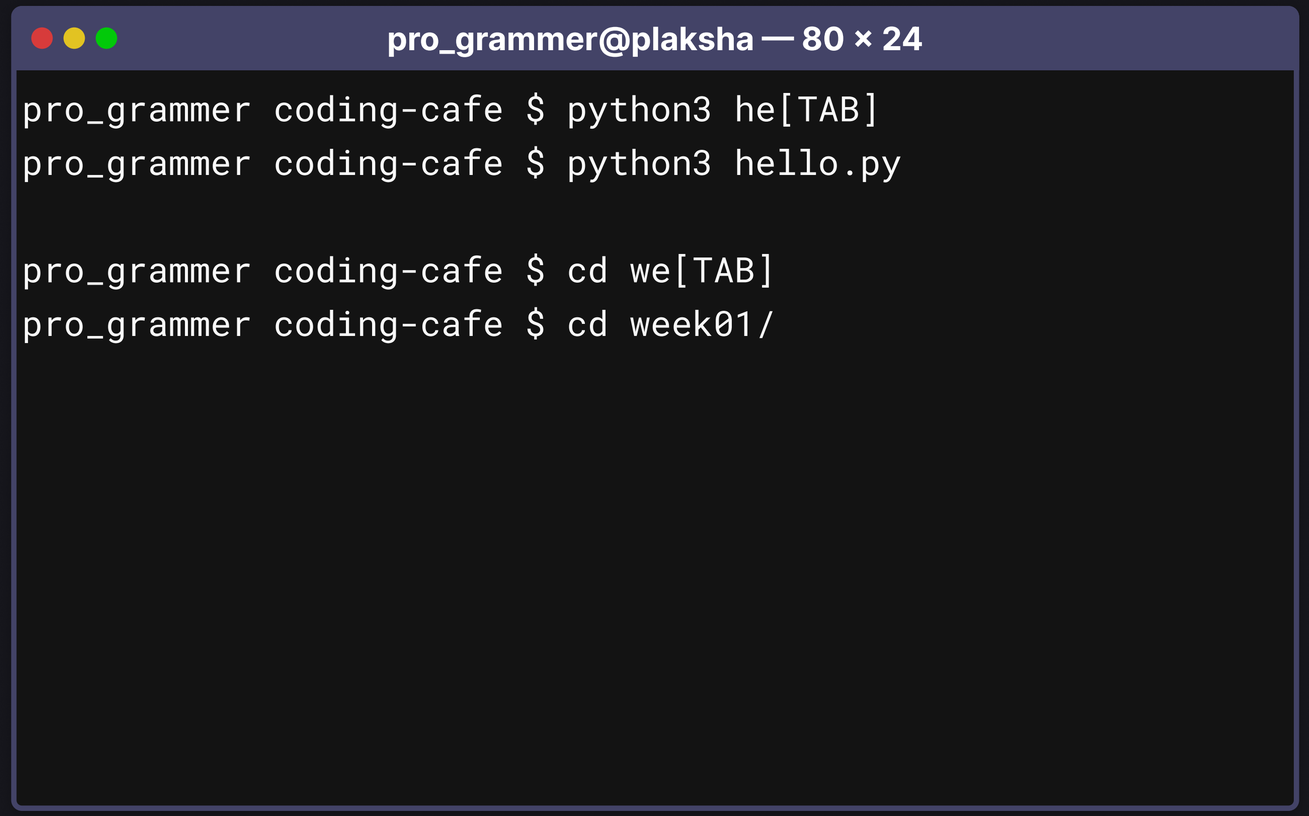
Outline

1. Why a terminal?
2. Finding your way
3. Files and folders
4. How the shell finds things
5. Running Python from the shell
6. Installing things with pip
7. Working faster

Getting unstuck.

action / error message	effect
`Ctrl + C`	stop whatever is running
`Ctrl + D`	end input, or leave the REPL
`clear`	wipe the screen, keep the session
`q`	leave most things that take over your screen
`man <cmd>`	the manual for any command - 1 to get out again
`“command not found”`	check the spelling, then run which; it is usually a PATH issue
`“No such file or directory”`	run pwd and ls; you are not where you think you are

Never type a whole filename.

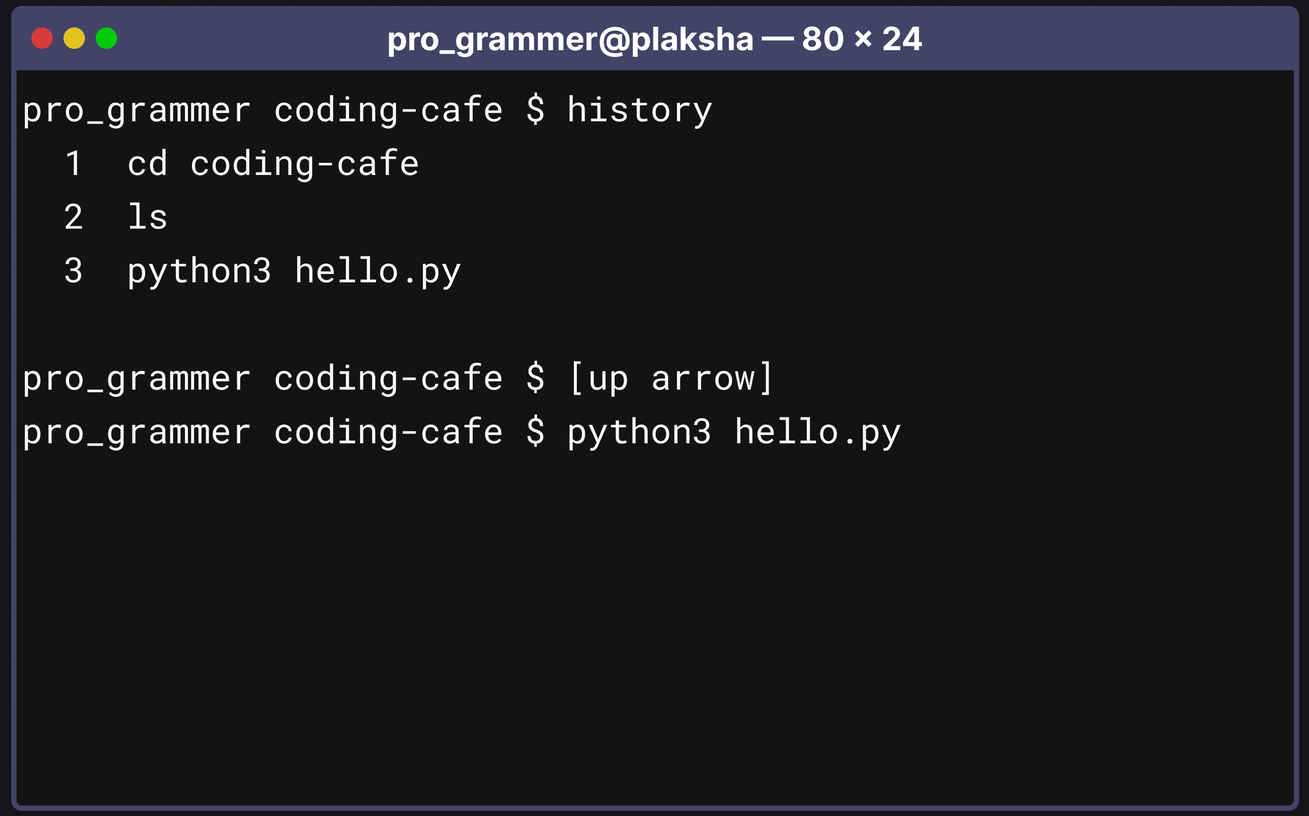


Type the first few letters of a filename, press Tab.

The shell finishes it for you.
Press Tab twice and it shows every option.

This is not a party trick - it is how people actually use a terminal, and it kills typos before they happen while also saving you a few keystrokes.

You already typed it once.



`Up arrow` - the last command, and the one before that

`Ctrl + R` - search backwards through what you have typed

`history` - the whole list

The shell remembers so you do not have to.

The whole set, on one slide.

program	outcome	extensions
`ls`	what is here?	-l (all) -a (all)
`cd`	go somewhere	.. (up) ~ (home) - (back)
`mkdir`	make a folder	-p (the whole chain)
`touch`	make an empty file	
`cp`	copy	
`mv`	move, or rename	
`rm`	delete	-r (and everything inside)

program	outcome
`pwd`	where am I?
`which`	where a program lives
`echo $PATH`	the folders the shell searches
`man`	the manual for all programs
`python3 file`	run a script
`python3`	open the REPL
`pip install`	get somebody else’s published code

If you want more than this.

dev-tools.mo8it.com - the terminal and Git, done properly
missing.csail.mit.edu - MIT’s Missing Semester

So, what are you actually talking to when you type into a terminal?

A shell. A program running on your machine, whose whole job is to turn your text into an action.

25 AUGUST 2026

fin